

Week 6 — Version Control and Git Flow

CSE5IDP Industry Development Project · Presenter: Scott Mann · Online cohort notes

Semester 2, 2026 · Milestone 3 opens · Thursday 20 August

Roughly 1h50, split into a version-control theory half, a live GitHub and VS Code demo half, and about 15 minutes of Q&A. This opens **Milestone 3**.

Housekeeping

- Leafy AI got firewall address space whitelisted for the internal Raspberry Pi, so the data deadlock from W5 is resolved from both directions. A technical session with the project owners ran Friday 2pm.
- The supplementary trade show platform was opened during Friday's lab (11:00–13:00) to get traffic through for the few teams still needing to complete it.
- **Scrum masters: upload your slide deck to the LMS before presenting** — that lets you be graded on the spot.

Scott flagged he is covering Milestone 3 topics 1 and 2 now, then jumping ahead to topic 4 (testing) because **testing must happen in every sprint, not as an end-of-semester afterthought**.

Agile process and Jira

Feedback drawn from the scrum master presentations so far:

- **Keep the board current, daily.** He specifically called out the “burn-down cliff” — a week's work all dragged into Done the morning the sprint closes. Tolerable for sprints 1–2, not beyond.
- **Story point accounting happens on parent tasks, not child tasks.** The parent has to be dragged into Done for the effort to flow through to the burn-down chart. Worth checking against how your team has structured epics and subtasks.
- Tie this back to the W5 role types: if someone in the team owns keeping everyone accountable for board updates, reporting gets easier and team motivation goes up, because progress is visible in real time rather than batched.
- Jira is mandated here, but the point is enacting agile correctly — tooling is a team or organisational decision later.

Version control (the core of the lecture)

The expectation stated plainly: **GitHub activity logs are the evidence of collaboration** in every remaining scrum master presentation — who committed what, how often, and what the branch structure looks like.

Framing. Version control is *distributed code collaboration*, not a backup system. It has backup-like properties but you do not do enterprise backup with Git.

Traceability as source of truth. When teams disagree about workload allocation — which Scott says happens every semester — staff look at the repository and activity logs. Related warning: do not have one person commit on behalf of others. You write the code, you commit the code.

The Git flow model

He gave an unusually prescriptive roadmap here.

Branch	Purpose
<code>main / master</code>	Sacred. Only clean, working, demonstrable point releases (1.1, 1.2, 1.3). Never day-to-day development.
<code>hotfix</code>	Short-lived, for critical bugs. Fix, test, merge to main as a point release; propagates forward.
<code>develop</code>	The aggregator for all sprint feature work. Mistaking this for main is “fundamental 101 wrong”.
<code>feature/*</code>	One per feature, mapped to your Jira board. Branch off develop, merge back to develop.

The flow per sprint: branch off `develop`, cut feature branches, unit test each feature, merge back to `develop`, run integration and system testing, release, then merge to `main` as a point release you can demo to the project owner.

Anti-patterns he called out:

- **Single-branch teams.** Six people committing to one branch guarantees conflicts. Having one branch does not tick the version control box.
- **Committing untested code.** “You don’t wanna be that developer.”
- **Going feature to main directly**, skipping develop and release testing.

Avoiding merge conflicts is a planning problem, not a Git problem. Scope which features and which parts of the code base belong to which team member *before* the feature branches are cut, so contributions stay compartmentalised. Comparing hundreds of conflicting lines every sprint is a productivity killer. When conflicts do happen, he would like you to resolve them as a team in IDP as a learning exercise — a code review on the diff, deciding which source of truth rolls forward.

Testing granularity: unit tests are the feature developer’s job at the function and method level; integration and system-level testing happens at merge and pre-release.

Repository hygiene (live demo)

He created a repo live and walked through it. The professional-standard checklist:

- **Private repo** for IDP (project owners will want the code private), which means you must explicitly invite every team member.
- **A real description**, roughly 150–350 characters. Empty or dummy descriptions reflect on the team.
- **README at root, in markdown** — headers, code blocks, images. Well-documented repos are the ones that stand out; go look at GitHub trending repos for structure and layout patterns, not just project ideas.
- **.gitignore** — track only code your team wrote, never vendored library or template code. He is unzipping 40-plus team repos at semester end and will not tolerate 1.5 GB archives.
- **Licence** chosen to reflect the project and the owner’s wishes; he demoed a licence-selection questionnaire.
- He **skips GitHub’s Copilot Jumpstart** on new repos — he wants full control over the initial structure and to know what is in the project before layering AI in later.

Handover format: a zip of the repository uploaded to the LMS (as an auditable snapshot) **plus** the repository URL, which gets passed to the project owner.

Tooling notes: terminal first, GUI second — understand what is happening underneath before abstracting it away. He drew an explicit parallel to using AI without understanding the generated code. GitHub Desktop, GitLens and VS Code’s source control panel were all mentioned. Any IDE without integrated version control should be replaced. The GitHub Student Developer Pack is worth signing up for with your student ID.

Commit frequency: a couple per day commercially, every couple of days in IDP is fine. Commit after testing, so what you push is defensible.

Career angle: a public GitHub repository is your portfolio in 2026 — the single strongest piece of evidence you can put in front of an employer, because it validates the CV rather than just asserting it.

Q&A — tender document (due today)

Several answers here are directly usable:

- **Budget calculation.** Baseline off your *actual* recorded hours from sprints 1 and 2 rather than guessing. The reference figure he had given previously: 6 students times roughly 15 hours per week times roughly \$40 per hour. Present **overall projected values only** — no per-student breakdown needed.
- **Deliverables section.** Do *not* attach individual names to tasks; it gets complicated and loses clarity. Use broad role categories instead — project coordinator, research lead, and so on.
- **Statement of work timeline.** Include the trade show as a milestone, since it was public-facing and is where you presented your prototype. Do *not* include the tender document itself — it is internal assessment and the project owner has no visibility over it.
- **Template versus rubric.** The supplied template is a baseline he described as a car chassis: everyone gets the same chassis, you fit your own engine and transmission. Adding unique sections to satisfy the rubric is, in his words, “my sneaky plan” — the purpose of a tender is to show your team is better than the other team, so customisation is the point.
- **Limitations and restrictions.** One of the marking criteria is identifying which team understands the project best, and the signal for that is a team that seriously addresses constraints and limitations. But keep it project-focused — do not volunteer things that make your team look unavailable (second jobs was the example). The purpose is to win the contract.
- **GitHub accounts.** Personal or student email are both fine; only the Student Developer Pack needs the student ID. All that matters is that the whole team and staff have repository access.

Also a career tangent on the project management path — no one goes into PM straight from university, you need three to five years showing escalating responsibility, and Scott promised to produce a “learning journal” template before semester end for capturing per-project evidence.

Next week

Agile metrics and methodologies — process, project and product metrics, trend lines across sprints (story point velocity, errors found per sprint) as evidence of continuous improvement. Plus the testing deep dive with interactive demos.